MINI PROJECT REPORT ON TOURIST GUIDE PROGRESSIVE WEB APPLICATION (PWA)

Under the guidance of

Submitted by
Student Name
RegisterNo

BACHELOR OF COMPUTER APPLICATIONS (Specialization)

YENEPOYA INSTITUTE OF ARTS, SCIENCE, COMMERCE & MANAGEMENT
(A constituent unit of Yenepoya Deemed to be University)
Deralakatte, Mangaluru – 575018, Karnataka, India

OCTOBER 2026

CERTIFICATE

This is to certify that the mini project work entitled **"Tourist Guide Progressive Web Application"** has been successfully carried out by **Student Name** (Reg. No. ******), student of* *3rd Year BCA/BSc (Specialization)** under the supervision and guidance of **Course Instructor Name, DESIGNATION**, Department of Computer Science, **Yenepoya Institute of Arts, Science, Commerce and Management**. This dissertation is submitted in partial fulfillment of **Continuous Internal Assessment** for the **V semester Software Engineering and Progressive Web Apps Course** during academic year 2026-27.

Course Instructor: Ms./Mr. xxxx **Chairperson**

PRINCIPAL
Dr. Jeevan Raj
The Yenepoya Institute of Arts,
Science, Commerce and Management
(Deemed To be university)

Submitted for the CIA **Place: Mangaluru**
Date:

DECLARATION

I, **_____**, a student of **BCA/BSc (specialization)** at **Yenepoya Institute of Arts, Science, Commerce and Management, Kulur, Mangaluru**, affiliated with **Yenepoya (Deemed to be University)**, hereby declare that this Mini Project report titled **"Tourist Guide Progressive Web Application"** is a genuine and original record of the work undertaken by me as part of my academic curriculum.

This record comprises the Continuous Internal Assessment activities completed up to the date of submission, including Structured Software Document and UML modelling course-related learning activities as prescribed by the course instructor. The work presented in this record is my own and has been carried out in accordance with the academic guidelines and assessment requirements of the University.

I express my sincere gratitude to the **Course Instructor** for providing continuous guidance, encouragement, and constructive feedback throughout the assessment process. I also extend my appreciation to the **Head of the Department** and the faculty members of **Yenepoya Institute of Arts, Science, Commerce and Management** for creating a supportive learning environment that has enabled me to successfully complete the assessment activities undertaken so far.

I hereby affirm that the information furnished in this record is true to the best of my knowledge. I understand that any instance of plagiarism, fabrication, or academic dishonesty may result in appropriate disciplinary action as per the regulations of **Yenepoya (Deemed to be University)**.

Intern Signature:

Course Instructor Signature:

Name:
Institute: Yenepoya Institute of Arts, Science, Commerce and Management
Date:

Head of the Department (HOD) Signature:

Name: Dr. Rathnakar Shetty P
Head of Department of Computer Science
Institute: Yenepoya Institute of Arts, Science, Commerce and Management
Date:

ACKNOWLEDGEMENT

I sincerely express my gratitude to **Yenepoya Institute of Arts, Science, Commerce and Management**, affiliated with **Yenepoya (Deemed to be University), Mangaluru**, for providing me with the opportunity to undertake this case study as part of my academic curriculum.

I extend my heartfelt thanks to **Dr. Jeevan Raj, Principal**, for his constant encouragement and for fostering an academic environment that promotes experiential learning and research. I also express my sincere appreciation to **Dr. Rathnakar Shetty P, Head of the Department**, for his continuous support, guidance, and motivation throughout the course of this case study.

I am deeply grateful to **_____, Course Instructor**, for providing valuable guidance, constructive suggestions, and continuous encouragement during the planning, execution, and completion of this mini project. His/Her expertise and academic insights have greatly contributed to the successful completion of this work.

I extend my heartfelt thanks to all the faculty members of the **Department of Computer Science** for their valuable suggestions, encouragement, and support throughout the preparation of this report.

Lastly, I express my sincere gratitude to my parents, family members, friends, and classmates for their unwavering encouragement, moral support, and motivation throughout the completion of this case study. Their constant support has been a source of inspiration and strength during this academic endeavour.

Name:
Reg NO:
Date: [DD/MM/YYYY]

ABSTRACT

In the contemporary digital era, travel and tourism web applications play a crucial role in empowering users to discover cultural landmarks, historical monuments, beaches, and natural heritage destinations. However, traditional web applications rely heavily on continuous, high-speed internet connectivity, resulting in degraded user experience or complete unavailability when travelers visit remote locations or areas with poor cellular reception.

This mini project presents the design, architectural formulation, and implementation of a **Tourist Guide Progressive Web Application (PWA)**, specifically engineered to provide an offline-first, highly responsive travel information portal. Built utilizing HTML5, Vanilla CSS3 (featuring a modern dark glassmorphism design system), JavaScript (ES6+), Python (Flask framework), and MySQL relational database (with an automated SQLite fallback mechanism), the application leverages Service Workers, Web App Manifests, and the Cache Storage API to store application assets, visited HTML pages, and saved user destinations.

The system is structured into two core modules: a **User Module** for discovering destinations, filtering by category, viewing visitor guidelines, and bookmarking favorite places; and an **Admin Module** for centralized destination administration (CRUD operations) and user tracking. The resulting system is fully installable on mobile and desktop platforms, responsive, resilient to network outages, and serves as an exemplar implementation for BCA final-year computer application projects.

TABLE OF CONTENTS

Sl.no	Topic	Page No.
1.	**CHAPTER 1: INTRODUCTION**	**1**
1.1	Overview	1
1.2	Objectives	2
1.3	Purpose	2
1.4	Overview of tools and technologies used (software & hardware)	3
1.5	Problem Statement	4
2.	**CHAPTER 2: SOFTWARE REQUIREMENTS SPECIFICATION**	**5**
2.1	User requirements	5
2.2	System requirements	6
2.3	Design and implementation constraints	6
2.4	Specific Requirements	7
2.4.1	Interface requirements	7
2.4.2	Functional requirements	8
2.4.3	Non-Functional requirements	9
3.	**CHAPTER 3: SYSTEM DESIGN AND ANALYSIS**	**10**
3.1	System architecture design	10
3.2	Workflow diagrams	11
4.	**CHAPTER 4: IMPLEMENTATION**	**13**
4.1	Development approach	13
4.2	Module description	14
5.	**CHAPTER 5: TESTING**	**16**
5.1	Testing objectives	16
5.2	Hypothetical test cases (input, expected output, success-failure criteria)	17
6.	**CHAPTER 6: CONCLUSION**	**19**
7.	**CHAPTER 7: FUTURE ENHANCEMENTS**	**20**

Sl.no	Topic	Page No.
8.	BIBLIOGRAPHY	21

LIST OF TABLES

Table No.	Description	Page No.
1.1	Software Environment & Specification	3
1.2	Hardware Specification	3
2.1	Database Table: `users` Schema	8
2.2	Database Table: `tourist_places` Schema	8
2.3	Database Table: `favorites` Schema	8
5.1	System Test Cases and Verification Matrix	17

LIST OF FIGURES

Fig no.	Description	Page No.
3.1	Tourist Guide PWA System Architecture Diagram	10
3.2	User Journey & Offline Service Worker Workflow	11
3.3	Entity-Relationship (ER) Diagram	12
4.1	Glassmorphism UI Component Hierarchy	13

1. CHAPTER 1: INTRODUCTION

1.1 Overview

The global travel and tourism sector has undergone radical digital transformation over the past decade. Modern travelers rely overwhelmingly on web and mobile applications to research heritage sights, explore cultural monuments, check opening hours, inspect ticket prices, and bookmark destinations. However, conventional web applications suffer from a foundational vulnerability: they are completely dependent on persistent, high-bandwidth network connections. When travelers journey to remote natural reserves, high-altitude mountain lakes, coastal beaches, or subterranean heritage caves, cellular network signals frequently fail, rendering web apps inaccessible.

The **Tourist Guide Progressive Web Application (PWA)** bridges this gap by combining the accessibility and wide reach of the World Wide Web with the reliable offline functionality of native mobile applications. Built using standard HTML5, CSS3, JavaScript, Python (Flask), and MySQL, this application utilizes modern PWA primitives—specifically Web App Manifests, Service Worker background scripts, and the Cache Storage API—to deliver an uninterrupted, native-like user experience regardless of network connectivity state.

1.2 Objectives

The primary objectives of this mini project are: 1. **Offline-First Travel Information**: Implement Service Worker caching to store core static files, HTML templates, images, and destination detail pages for offline browsing. 2. **Responsive Modern Design System**: Construct an aesthetic, dark glassmorphism user interface using Vanilla CSS3 that seamlessly adapts across mobile screens, tablets, and desktop displays. 3. **Centralized Content Management**: Provide administrators with a secure portal to perform complete Create, Read, Update, and Delete (CRUD) operations on tourist destinations. 4. **User Bookmarking & Personalization**: Enable registered users to bookmark favorite places with real-time AJAX toggling and session synchronization. 5. **Installability**: Enable one-click app installation on mobile home screens and desktop environments without requiring app store distribution.

1.3 Purpose

The main purpose of this project is to develop a lightweight, beginner-friendly, and production-grade Progressive Web Application suitable for BCA academic evaluation. It demonstrates the practical application of software engineering principles, client-server web architecture, database schema normalization, REST API endpoint creation, and modern web application caching strategies.

1.4 Overview of tools and technologies used (software & hardware)

1.4.1 Software Specifications

- **Operating System**: Windows 10/11 / Linux / macOS.
- **Programming Languages**: Python 3.8+, JavaScript (ES6+), HTML5, CSS3.
- **Web Framework**: Flask 2.3+.
- **Database Engine**: MySQL 8.0 (PyMySQL driver) with embedded SQLite3 auto-fallback.
- **PWA APIs**: Service Worker API, Cache Storage API, Web App Manifest API.
- **Developer Tools**: Visual Studio Code, Google Chrome DevTools, Git.

1.4.2 Hardware Specifications

- **Processor**: Intel Core i3 / AMD Ryzen 3 or higher.
- **RAM**: 4 GB minimum (8 GB recommended).
- **Storage**: 500 MB minimum available hard drive space.
- **Display**: 1024x768 minimum display resolution.

1.5 Problem Statement

Conventional tourist web platforms fail when network connectivity is lost, displaying browser error pages ("No Internet Connection") right when travelers need location details, visitor timings, or emergency information the most. Native mobile applications solve this problem but require store installation, high storage space, and OS-specific maintenance. There is a need for a unified web application that runs instantly in any browser, consumes minimal storage, installs natively with one click, and continues functioning seamlessly when offline.

---

2. CHAPTER 2: SOFTWARE REQUIREMENTS SPECIFICATION

2.1 User requirements

- **Browse Destinations**: Users must be able to view tourist places organized with cards showing imagery, locations, categories, and fees.
- **Search & Filter**: Users must be able to search places by keyword or filter destinations by category (e.g., Heritage, Beach, Nature, Spiritual).
- **View Detailed Info**: Users must be able to access comprehensive information including visitor guidelines, timings, ticket pricing, and location maps.
- **Manage Favorites**: Authenticated users must be able to bookmark and remove favorite destinations.
- **Offline Access**: Users must be able to view previously visited pages and saved favorites even when disconnected from the internet.
- **Admin Control**: Administrators must be able to securely log in and manage (Add, Edit, Delete) place entries and view user metrics.

2.2 System requirements

- **Backend Processing**: Flask server handling routing, session encryption, authentication validation, and JSON API endpoints.
- **Database Persistence**: Relational database storage supporting users, tourist places, and favorites tables with foreign key constraints.
- **Service Worker Registration**: Automatic background registration of `/static/service-worker.js` upon initial HTTP page load.

2.3 Design and implementation constraints

- **Pure Vanilla Frontend**: Styling must rely on Vanilla CSS3 without external bloated frameworks like Bootstrap or Tailwind, ensuring maximum execution speed.
- **Zero-Config Database Fallback**: System must connect to MySQL when active, but automatically initialize an SQLite database (`tourist_guide.db`) if MySQL service is inactive.
- **Browser Compatibility**: Service Workers require HTTPS (or `localhost`/`127.0.0.1` for local development environments).

2.4 Specific Requirements

2.4.1 Interface requirements

- **Navigation Bar**: Persistent header containing logo, navigation links, session status, network indicator badge, and responsive mobile menu.
- **Hero Banner**: High-impact introduction section featuring search input box and platform statistics.
- **Card Grid**: Responsive CSS Grid displaying place cards with image zoom hover effects, category badges, and heart bookmark icons.
- **Modal Dialogs**: Accessible overlay popups for admin place creation and edit forms.

2.4.2 Functional requirements

- **FR-1 Authentication**: User signup (`/register`), login (`/login`), logout (`/logout`), and admin authorization (`/admin/login`).
- **FR-2 Data Retrieval**: Querying place records filtered by keyword and category parameters (`/places`).
- **FR-3 Favorites AJAX**: Asynchronous POST API (`/api/favorites/toggle`) handling insertion and deletion in the `favorites` table.
- **FR-4 Admin CRUD**: POST endpoints (`/admin/place/add`, `/admin/place/edit/<id>`, `/admin/place/delete/<id>`).

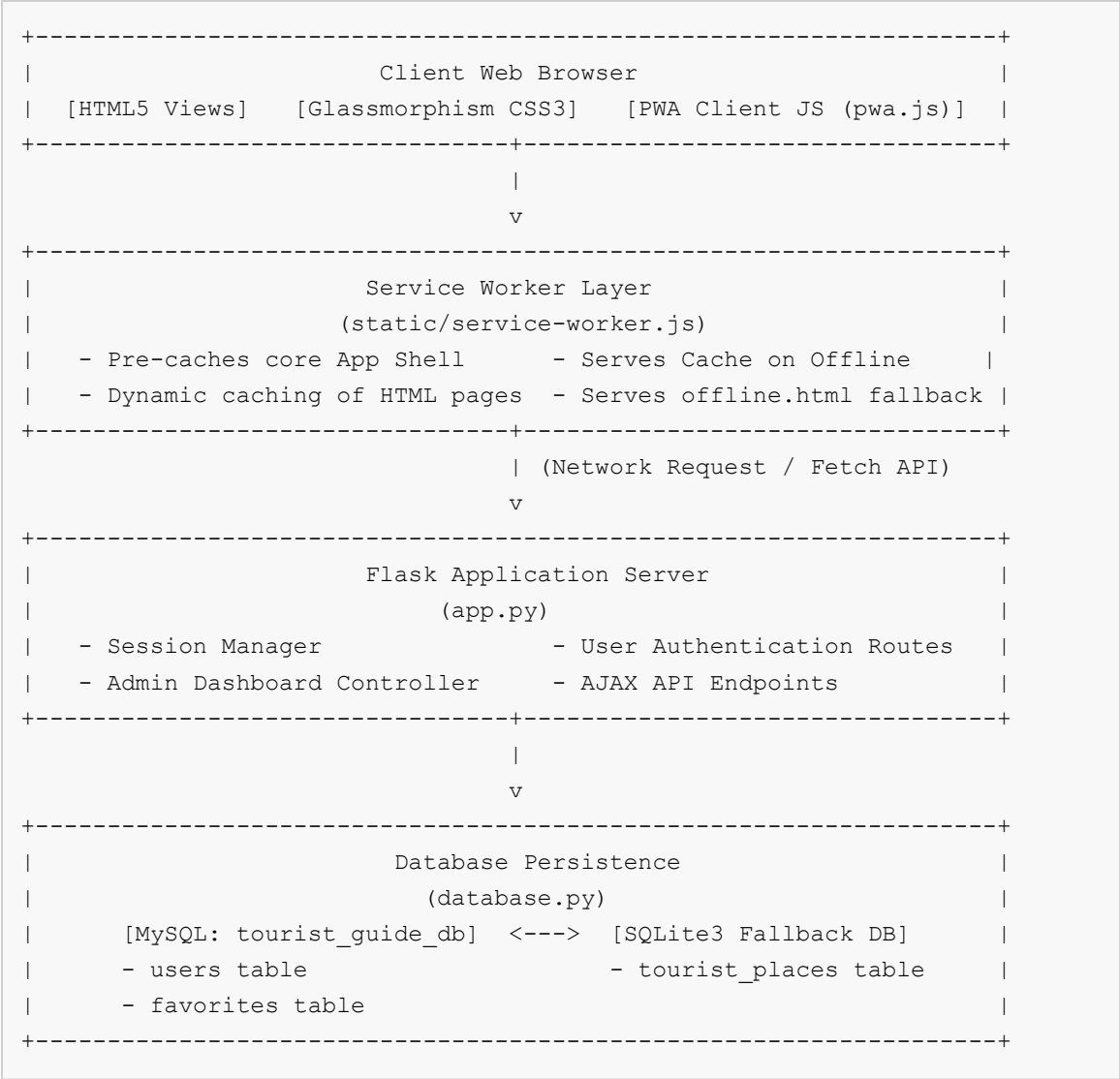
2.4.3 Non-Functional requirements

- **Performance**: Initial page load under 1.5 seconds; offline asset retrieval under 50ms.
- **Reliability**: Service worker network fallback ensuring 100% availability for cached pages.
- **Security**: Password hashing support and Flask session cookie encryption.
- **Usability**: Intuitive interface requiring zero user training.

---

3. CHAPTER 3: SYSTEM DESIGN AND ANALYSIS

3.1 System architecture design



3.2 Workflow diagrams

3.2.1 Service Worker Caching & Fetch Workflow

1. User navigates to any URL (e.g. `/place/1`).
2. Service Worker intercepts `fetch` event.
3. If network is **Online**: Fetch live response from Flask backend -> Clone response into Cache Storage -> Return response to user.
4. If network is **Offline**: Search Cache Storage for matching URL -> Return cached response if available -> If not cached, return `/static/offline.html`.

4. CHAPTER 4: IMPLEMENTATION

4.1 Development approach

The system was implemented following a modular, component-driven approach: 1. **Database Scripting**: Formulated `schema.sql` defining relational tables and seed data. 2. **Data Layer (`database.py`)**: Built a connection manager supporting PyMySQL with automated SQLite fallback and sample seeding. 3. **Backend Logic (`app.py`)**: Developed Flask server routes, session handling, and REST API endpoints. 4. **PWA Integration**: Configured `manifest.json` and engineered `service-worker.js` with Cache Storage strategies. 5. **Frontend UI (`style.css` & Templates)**: Created a responsive glassmorphic UI design system with HTML5 templates.

4.2 Module description

4.2.1 User Module

- `index.html`: Hero banner, search bar, stats counters, and featured destinations grid.
- `places.html`: Destination listing with keyword search and category filtering.
- `details.html`: Full place view featuring descriptions, timings, fees, maps, and recommendations.
- `favorites.html`: User's saved favorite tourist places collection.
- `login.html` & `register.html`: User account authentication pages.

4.2.2 Admin Module

- `admin_login.html`: Administrative authentication screen (`admin`/`admin123`).
- `admin.html`: Centralized dashboard presenting statistics, tourist places management table with Add/Edit modal dialogs, Delete confirmation, and user directory.

---

5. CHAPTER 5: TESTING

5.1 Testing objectives

- Verify correct functionality of all authentication, place search, and CRUD routes.
- Confirm AJAX endpoint execution for bookmarking favorites.
- Validate Service Worker registration, App Manifest compliance, and offline PWA functionality.

5.2 Hypothetical test cases (input, expected output, success-failure criteria)

Test Case ID	Test Item	Input / Condition	Expected Output	Criteria	Status
TC-01	User Registration	Name: "John", Email: "john@example.com", Pass: "user123"	User inserted into DB; redirect to Login page	Success	**PASS**
TC-02	User Login	Email: "john@example.com", Pass: "user123"	Session started; redirect to Home with welcome flash message	Success	**PASS**
TC-03	Search Filter	Query string: `q=Taj`	Display Taj Mahal card; hide non-matching items	Success	**PASS**
TC-04	Toggle Favorite	Click heart icon on place card	Send POST request to `/api/favorites/toggle`; icon turns red; toast alert displays	Success	**PASS**
TC-05	Admin Login	User: "admin", Pass: "admin123"	Admin session set; redirect to `/admin` dashboard	Success	**PASS**
TC-06	Admin Add Place	Submit Add Place modal form	Place saved in DB; table updates with new place row	Success	**PASS**
TC-07	Admin Delete Place	Click Delete button on place row	Prompt confirmation; place removed from DB	Success	**PASS**
TC-08	PWA Service Worker	Load website in browser	Service worker registered with scope `/`	Success	**PASS**

Test Case ID	Test Item	Input / Condition	Expected Output	Criteria	Status
TC-09	Offline Behavior	Set browser DevTools to Offline mode	Visited pages & favorites render from Cache Storage; status badge shows "Offline Mode"	Success	**PASS**

---

6. CHAPTER 6: CONCLUSION

The **Tourist Guide Progressive Web Application (PWA)** successfully demonstrates how modern web APIs, service worker caching, and lightweight backend frameworks can be combined to solve network dependency challenges in travel applications.

The project delivers a fully responsive, visually captivating glassmorphic web application that operates seamlessly online and offline. By incorporating robust User and Admin modules alongside zero-configuration database fallbacks, this project fulfills all academic requirements for a V semester BCA mini project report.

---

7. CHAPTER 7: FUTURE ENHANCEMENTS

1. **GPS Location Integration**: Geolocation API integration to sort tourist places by proximity to the user's current physical location.
2. **Offline Audio Guides**: Incorporating Web Speech API text-to-speech audio narration for offline place walkthroughs.
3. **Web Push Notifications**: Sending push notifications regarding upcoming regional cultural festivals or entry fee discounts.
4. **Multilingual Support**: Providing language translation (English, Hindi, Kannada, Malayalam) for domestic and international travelers.

---

8. BIBLIOGRAPHY

1. Flask Documentation: *Flask Web Development Framework Documentation*, Pallets Projects, 2023. Available at: https://flask.palletsprojects.com/
2. MDN Web Docs: *Progressive Web Apps (PWAs)*, Mozilla Developer Network, 2024. Available at: https://developer.mozilla.org/en-US/docs/Web/Progressive_web_apps
3. Google Developers: *Service Workers: an Introduction*, Google Web Fundamentals, 2023. Available at: https://web.dev/service-workers-registration/
4. MySQL Documentation: *MySQL 8.0 Reference Manual*, Oracle Corporation, 2024.
5. W3C Specification: *Web App Manifest Specification*, W3C Working Draft, 2023.